Disciplina: FACOM31701 - Trabalho de Conclusão de Curso 1

Professor: Prof. Dr. Rafael Dias Araújo

Estudante: Rafael Lopes Fraga

Orientador(a): Prof. Dr. Paulo Rodolfo da Silva Leite Coelho

Tipo de trabalho: Projeto de Pesquisa Tradicional

Entrega 2 (E2) - Visão geral sobre a fundamentação teórica

1) Identifique, juntamente com o seu orientador(a), quais são os principais conceitos teóricos associados ao seu trabalho. Exemplo 1: se o tema do trabalho é o uso de algoritmos genéticos para análise de risco de crédito, então os principais conceitos são: algoritmos genéticos e análise de risco de crédito. Exemplo 2: se o tema do trabalho é o desenvolvimento de um chatbot para atender a comunidade UFU, então os principais conceitos são: processamento de linguagem natural, chatbots e formas de atendimento à comunidade comumente feitas pelas universidades.

2) Escreva sobre cada um dos conceitos identificados no item anterior. Use a seguinte estratégia: i) forneça as principais definições; ii) apresente um breve histórico; iii) discuta os avanços recentes da área; e iv) mostre alguns exemplos relevantes e ilustrativos. Para todos os casos, incluir as referências consultadas. O objetivo aqui é ter um rascunho da Seção 2 - Fundamentação Teórica. Portanto, é de suma importância analisar outras monografias para entender o que deve ou não ser escrito nesta seção. Outras referências bibliográficas também poderão ser consultadas (por exemplo, artigos científicos e livros).

3) Ao final desta entrega, declare o uso ou não de Inteligência Artificial para produção deste trabalho.

Observações:

1. No caso do **relatório técnico profissional**, o texto a ser apresentado será i) um detalhamento do ambiente de atuação (empresa/instituição onde se trabalha), segmento de atuação, quais são as suas atribuições e responsabilidades dentro da empresa; ii) a definição dos conhecimentos necessários para a execução do trabalho e as restrições técnicas e operacionais do ambiente; iii) os padrões e boas práticas adotados pela equipe para realização das atividades profissionais; iv) o projeto que irá desenvolver destacando seu enquadramento e relevância no contexto empresarial/institucional e acadêmico. Para este modelo, deverá ser entregue também um termo de ciência e

concordância para desenvolvimento do trabalho, assinado pelo representante legal e/ou chefia imediata na empresa onde trabalha (ver modelo anexado à entrega).

2. No caso do **desenvolvimento de um software**, crie seções para i) estudo preliminar sobre a viabilidade da proposta, incluindo relevância, diferenciais e motivações, ii) as tecnologias específicas que se pretende utilizar com breve apresentação , histórico e justificativas para a escolha, iii) os principais princípios, técnicas e conceitos de engenharia de software a serem aplicados no desenvolvimento; iv) seções descritivas relacionadas ao domínio/subdomínios envolvidos na aplicação e necessárias para o entendimento da solução.

3. No caso de um **plano de negócios**, descreva: i) o domínio da aplicação e negócio a ser abordado; ii) uma versão inicial do produto e/ou serviço que será implementado no plano de negócios contendo objetivos, motivações e características; iii) benefícios, diferenciais, limitações e custos envolvidos; iv) o estágio atual de desenvolvimento, expectativas e planejamento preliminar.

4. É de suma importância que todos os parágrafos escritos sejam coesos. Ou seja, o texto como um todo precisa fazer sentido;

5. Use o formato de citações do Latex para apresentar as referências em todo o texto. Deve ser entendido como as citações funcionam no Latex, em particular com o BibTeX.

6. O posicionamento de citações no texto e orientações da ABNT serão discutidos nas aulas. Lembre-se que no modelo fornecido para a entrega 1, além do arquivo `main.tex` utilizado para redigir seu texto, há um arquivo chamado `ref.bib` com alguns exemplos de referências. Um tutorial sobre o overleaf está disponível em (OVERLEAF, 2025).

7. Junto da entrega, o aluno deverá anexar um comprovante de concordância do seu orientador quanto ao texto sendo submetido. O comprovante de concordância poderá ser um e-mail do seu orientador, onde ele manifesta autorização para submeter o texto, ou uma assinatura eletrônica no PDF submetido feita pelo orientador, preferencialmente pelo sistema GovBr (ou outra plataforma de assinaturas conhecida e com validação pública).

1 RASCUNHO DA FUNDAMENTAÇÃO TEÓRICA

Tema do trabalho: Padrão Saga em Arquiteturas de Microsserviços: Um Estudo Comparativo entre os Modelos Coreografado e Orquestrado.

Os conceitos selecionados a partir do recorte apresentado na E1 são: arquitetura de microsserviços e autonomia dos dados; transações e consistência distribuída; padrão Saga e compensações; coordenação coreografada e orquestrada; mecanismos de recuperação e comunicação confiável; e métricas de recuperação e desempenho. Essa organização constitui uma proposta de fundamentação a ser apreciada pelo orientador. O eixo principal permanece a recuperação de falhas e a consistência, enquanto o desempenho constitui o eixo complementar. As definições são acompanhadas de antecedentes históricos, desenvolvimentos da literatura e exemplos ligados ao cenário de liquidação de ordens.

1.1 Arquitetura de microsserviços e autonomia dos dados

A arquitetura de microsserviços estrutura uma aplicação em serviços organizados em torno de capacidades de negócio, que se comunicam por interfaces e podem ser implantados de maneira independente. O tamanho do código, isoladamente, não caracteriza um microsserviço: importam os limites de responsabilidade e a autonomia de evolução. Na sistematização de padrões apresentada por Richardson (2018), a decomposição em serviços é examinada junto com comunicação e gerenciamento de dados, pois a separação de responsabilidades também introduz desafios de coordenação.

No padrão *Database per Service*, os dados de cada serviço são privados e devem ser acessados pelos demais por interfaces, em vez de consultas diretas às suas tabelas. Essa separação reduz o acoplamento entre esquemas, mas torna mais complexas as operações que atualizam dados de vários serviços (RICHARDSON, 2018). A autonomia dos dados é uma propriedade de acesso e responsabilidade; o padrão não exige, por definição, uma máquina física exclusiva para cada banco. Neste trabalho, contudo, estão previstas instâncias independentes de PostgreSQL por serviço de domínio e para o estado do coordenador, conforme a delimitação da E1.

Como exemplo, o serviço de conta é responsável pelos saldos e suas reservas, enquanto o serviço de custódia controla as posições e reservas de ativos. Uma solicitação que depende dos dois serviços precisa coordenar seus efeitos sem permitir acesso cruzado aos bancos. A separação permite alterações locais, mas introduz comunicação pela rede,

atrasos e situações em que um serviço conclui uma etapa e outro fica indisponível. Portanto, independência de implantação não significa ausência de dependências durante um processo de negócio (RICHARDSON, 2018).

A discussão sobre esses desafios aparece nos estudos comparativos de Kristianto e Zahra (2021) e Marques (2024), que examinam formas de coordenar serviços e seus compromissos de desempenho, manutenção e tratamento de falhas. Esses trabalhos ajudam a situar o problema desta pesquisa: a autonomia dos serviços precisa ser conciliada com a execução de operações que dependem de mais de um deles. Seus resultados serão discutidos na seção sobre coordenação coreografada e orquestrada.

1.2 Transações e consistência em operações distribuídas

1.2.1 TRANSAÇÕES LOCAIS, PROPRIEDADES ACID E CONFIRMAÇÃO DISTRIBUÍDA

Uma transação agrupa operações sobre dados em uma unidade de execução. As propriedades ACID correspondem à atomicidade, que permite confirmar ou abortar suas alterações como uma unidade; à consistência, que exige preservar as restrições de integridade definidas; ao isolamento, que controla a interferência entre execuções concorrentes; e à durabilidade, que preserva alterações confirmadas. O nível de isolamento efetivamente configurado pode admitir comportamentos diferentes daqueles de uma execução serial. Uma transação local assegura essas propriedades no âmbito dos recursos que administra, sem estendê-las automaticamente aos bancos dos demais serviços (GRAY, 1978; RICHARDSON, 2018).

O protocolo *Two-Phase Commit* (2PC), descrito na exposição clássica de Gray (1978), coordena a confirmação atômica entre participantes. Na primeira fase, o coordenador solicita a preparação e coleta os votos; na segunda, comunica a decisão de confirmar ou abortar. Um participante que votou favoravelmente deve preservar condições para cumprir a decisão. Dependendo do mecanismo de concorrência, recursos podem permanecer retidos enquanto essa decisão não é conhecida. O protocolo de confirmação, entretanto, não substitui os mecanismos de isolamento: atomicidade e isolamento são propriedades distintas (GRAY, 1978).

Historicamente, a coordenação de transações distribuídas antecede a arquitetura de microsserviços. A questão retomada nesse contexto é como manter regras de negócio quando os dados pertencem a serviços com ciclos de execução independentes. Richardson (2018) apresenta Saga como uma alternativa baseada em confirmações locais e compensações. O 2PC é utilizado nesta fundamentação para esclarecer a diferença entre as garantias, sem compor uma terceira variante experimental nem ser considerado universalmente inferior.

1.2.2 CONSISTÊNCIA EVENTUAL E REGRAS DE NEGÓCIO

Na coordenação por Saga, alterações locais confirmadas podem ficar visíveis antes da conclusão do processo completo. A consistência eventual expressa a expectativa de alcançar um estado final compatível com as regras da aplicação após o processamento das etapas e das recuperações necessárias. Isso depende de condições como entrega das mensagens, disponibilidade dos participantes e execução correta das ações; não decorre apenas da passagem do tempo. Além disso, consistência eventual entre serviços não implica ausência de transações ACID dentro de cada serviço (RICHARDSON, 2018).

No cenário deste trabalho, as regras de consistência devem relacionar os estados da ordem, das reservas e da liquidação. Como exemplo conceitual, uma ordem cancelada antes da efetivação não deve manter reservas sem finalidade, e uma ordem concluída deve possuir os efeitos correspondentes registrados. Esses critérios não exigem que todos os bancos sejam alterados no mesmo instante, mas exigem que estados intermediários sejam identificáveis e tenham tratamento definido. O exemplo delimita regras a verificar no cenário estudado, sem representar normas de liquidação de uma infraestrutura financeira real.

1.3 Padrão Saga e transações compensatórias

1.3.1 ORIGEM, DEFINIÇÃO E RECUPERAÇÃO

Garcia-Molina e Salem (1987) propuseram Sagas no contexto de transações de longa duração, cujos recursos poderiam permanecer ocupados por períodos extensos. A proposta permite decompor uma transação longa em transações menores, intercaláveis com outras execuções, e tratar execuções parciais por meio de compensações. O artigo é a referência fundacional do conceito; não constitui uma comparação experimental entre os modelos modernos de coordenação coreografada e orquestrada.

Na aplicação a microsserviços, uma Saga é uma sequência coordenada de transações locais que implementa uma operação de negócio distribuída. Cada etapa confirma suas próprias alterações. Se uma condição impedir a continuação, transações compensatórias tratam os efeitos das etapas já concluídas. Em uma sequência linear na qual a etapa T_{k+1} falha antes de confirmar, uma recuperação possível é executar as compensações em ordem inversa:

$$T_1, T_2, \ldots, T_k, \quad C_k, C_{k-1}, \ldots, C_1.$$

Essa representação pressupõe etapas compensáveis e dependências compatíveis com essa ordem; etapas somente de leitura podem dispensar compensação (GARCIA-MOLINA; SALEM, 1987; RICHARDSON, 2018).

Compensar não equivale a executar um `ROLLBACK` global. Como as transações locais já foram confirmadas, a compensação é uma nova operação que corrige seus efeitos segundo a semântica do negócio. Por exemplo, liberar uma reserva de saldo pode compensar sua criação, mas não apaga o histórico de que a reserva existiu. A compensação também pode falhar e exigir recuperação posterior. Assim, a Saga oferece um modelo de coordenação, cuja correção depende das ações e das condições de recuperação implementadas (RICHARDSON, 2018).

1.3.2 ETAPAS COMPENSÁVEIS, PONTO DE DECISÃO E EXEMPLO ILUSTRATIVO

Richardson (2018) distingue etapas compensáveis, uma etapa de decisão denominada *pivot* e etapas posteriores que devem ser concluídas mediante novas tentativas. A etapa *pivot* marca a passagem para um trecho em que a estratégia deixa de ser desfazer as reservas anteriores e passa a ser completar o processo. Essa estrutura exige que falhas de negócio sejam resolvidas antes desse ponto e que as etapas posteriores tenham condições de conclusão quando as falhas temporárias cessarem. Não se trata de uma garantia de recuperação sob indisponibilidade permanente.

Como exemplo elaborado para esta fundamentação, uma ordem pode ser registrada como pendente, solicitar uma reserva de saldo e, depois, uma reserva de ativos. Se a reserva de ativos for rejeitada, a reserva de saldo deve ser liberada e a ordem deve receber um estado final de cancelamento. Se ambas forem aceitas, o fluxo pode prosseguir para a efetivação simplificada da liquidação e para as confirmações de estado. A divisão exata dessas confirmações será definida no modelo da aplicação. A organização preserva os quatro serviços de domínio previstos na E1: ordens, conta, custódia e liquidação.

O exemplo supõe que as reservas possam ser liberadas antes da efetivação. Uma liquidação já efetivada não deve ser tratada como reversível por simples analogia com uma reserva; seus efeitos exigiriam regras próprias. Por isso, a posição do ponto de decisão e as operações compensáveis precisam ser explicitadas antes da comparação. Na evolução do conceito de 1987 para o tratamento de microsserviços em Richardson (2018), a decomposição passa a ser discutida junto com coordenação, recuperação e concorrência, sem eliminar a necessidade de modelagem do domínio.

1.4 Coordenação coreografada e orquestrada

1.4.1 DEFINIÇÕES E EXEMPLOS DE INTERAÇÃO

Na coreografia, os participantes reagem a eventos e publicam novos eventos após suas transações locais, sem um coordenador central determinar toda a sequência. Um evento

comunica um fato ocorrido, como a criação de uma ordem ou a aceitação de uma reserva. No exemplo, o serviço de conta pode reagir ao evento `OrdemCriada`, reservar saldo e publicar `SaldoReservado`; custódia reage a esse fato para avaliar a reserva de ativos. Uma rejeição deve produzir a informação necessária para os participantes executarem as compensações previstas. A lógica do fluxo fica distribuída entre os produtores e consumidores desses eventos (RICHARDSON, 2018).

Na orquestração, um coordenador mantém o estado do processo, solicita ações aos participantes e decide as próximas etapas com base nas respostas. Um comando expressa uma ação solicitada, como `ReservarSaldo`, em vez de apenas anunciar um fato. O coordenador pode solicitar a reserva de saldo, depois a de ativos e, diante de uma rejeição, solicitar a liberação da reserva já aceita. O uso de comandos não obriga chamadas síncronas: Richardson (2018) apresenta orquestração com troca assíncrona de mensagens de comando e resposta.

Na variante planejada para este estudo, o coordenador será um componente técnico separado, responsável pelo fluxo e por seu estado persistente; as regras sobre saldo, ativos e liquidação continuam nos serviços de domínio. Essa separação pertence ao desenho do trabalho, pois o padrão de orquestração não exige que todo coordenador seja implantado como serviço independente. As duas variantes devem aplicar as mesmas regras de negócio e os mesmos critérios de recuperação, permitindo examinar a influência da forma de coordenação.

1.4.2 EVOLUÇÃO DOS ESTUDOS E COMPROMISSOS ARQUITETURAIS

Na literatura comparativa, Rudrabhatla (2018) avaliou coreografia e orquestração em um cenário de comércio eletrônico e observou menor tempo de execução na coreografia, inclusive em um fluxo com compensação. Esse resultado pertence à implementação e às cargas estudadas. As razões de tempo de resposta apresentadas para diferentes patamares de carga não devem ser interpretadas como valores absolutos de latência nem transferidas diretamente para outra aplicação.

Kristianto e Zahra (2021) compararam tempo médio de resposta, *throughput* e uso de CPU, variando o número de serviços, usuários concorrentes e instâncias. Seus resultados favoreceram a coreografia nessas métricas. Em uma alteração da sequência do fluxo, porém, relataram mudanças em seis APIs de quatro serviços na coreografia e em uma API do coordenador na orquestração. A análise mostra que custo de comunicação e facilidade de modificação são dimensões diferentes; não se pode inferir a recuperação sob falhas apenas a partir do tempo médio de resposta.

Aydin e Çebi (2022) discutiram vantagens de rapidez da coreografia e a adequação da orquestração a transações complexas. Suas conclusões sobre desempenho são qualitativas,

sem valores numéricos explícitos que permitam calcular ganhos ou comparar percentis. Em estudo posterior, Marques (2024) combinou uma implementação de referência com um levantamento entre 14 profissionais e discutiu o controle de erros e a manutenção na orquestração, bem como a independência dos serviços na coreografia. As observações sobre latência não constituem uma quantificação de seus percentis.

Esses estudos representam a evolução da discussão para além das definições dos padrões, mas utilizam cenários e procedimentos diferentes. Nos trabalhos examinados, não foram identificadas avaliações de latência de cauda que respondam ao recorte desta pesquisa. A centralização pode criar um ponto de falha se o coordenador não tiver recuperação adequada; a ausência desse componente na coreografia também não elimina dependências de serviços ou da infraestrutura de mensagens. Consequentemente, nenhuma das formas de coordenação assegura, por si só, melhor desempenho ou maior resiliência em qualquer cenário (RICHARDSON, 2018).

1.5 Comunicação confiável, concorrência e recuperação de falhas

1.5.1 MENSAGENS, IDEMPOTÊNCIA E PUBLICAÇÃO TRANSACIONAL

A comunicação assíncrona permite que o produtor de uma mensagem não dependa de sua execução imediata pelo consumidor. Um intermediário, ou *broker*, recebe e encaminha mensagens conforme sua configuração. Entretanto, confirmações perdidas ou interrupções durante o processamento podem provocar reentregas. Sob entrega de pelo menos uma vez, a mesma mensagem pode chegar mais de uma vez; essa propriedade não significa que seu efeito de negócio será automaticamente aplicado uma única vez (RICHARDSON, 2018).

Uma operação idempotente mantém o mesmo efeito de negócio quando repetida com a mesma identificação. Assim, repetir uma solicitação de reserva não deve reservar o valor novamente. Uma estratégia é registrar a identificação da operação processada na mesma transação local que altera os dados, permitindo reconhecer repetições. Identificar apenas a Saga pode ser insuficiente, pois várias operações distintas pertencem ao mesmo fluxo; é necessário distinguir o processo, a etapa e a solicitação. As compensações também devem tratar repetições para impedir, por exemplo, que uma reserva seja liberada duas vezes (RICHARDSON, 2018).

Outro problema ocorre quando a aplicação confirma uma alteração no banco e falha antes de publicar sua mensagem. No padrão *Transactional Outbox*, o serviço grava a alteração de negócio e uma mensagem pendente na mesma transação local; um processo posterior publica a mensagem no intermediário. Se a publicação ocorrer e a confirmação desse envio não for registrada, o publicador poderá repetir a mensagem. Portanto, a Outbox trata a relação entre gravação e publicação, mas continua exigindo consumidores

idempotentes (RICHARDSON, 2018).

Esses mecanismos já integram a sistematização de padrões de Richardson (2018) e complementam a coordenação por Saga ao tratar a comunicação entre suas etapas. Para este trabalho, eles constituem fundamentos para escolher políticas equivalentes nas duas variantes. A escolha de um mecanismo específico de publicação ou deduplicação pertence ao projeto da implementação, não decorre automaticamente da escolha de coreografia ou orquestração.

1.5.2 FALHAS DE NEGÓCIO, FALHAS TEMPORÁRIAS E CONCORRÊNCIA

Uma rejeição por saldo ou ativos insuficientes é uma falha de negócio: repetir a mesma solicitação não assegura sua aceitação. Uma indisponibilidade temporária pode, por outro lado, admitir nova tentativa. Já um *timeout*, entendido como o esgotamento do prazo de espera por uma resposta, informa apenas que essa resposta não foi recebida no prazo; a operação remota pode ter sido confirmada. Por isso, antes de repetir ou compensar, a coordenação precisa considerar identificadores, estado persistido e possíveis respostas tardias. Repetições devem ter política definida para evitar solicitações sem controle e preservar a coerência do fluxo (RICHARDSON, 2018).

A ausência de isolamento global permite interferências entre Sagas. Uma atualização perdida ocorre quando uma execução sobrescreve uma alteração de outra. Uma leitura de estado intermediário ocorre quando uma Saga usa alterações de outra ainda não concluída; nesse contexto, o dado pode estar confirmado no banco local, mas ainda sujeito a compensação no processo completo. Duas leituras feitas em etapas distintas também podem observar valores diferentes. Essas anomalias não são resolvidas apenas pelo uso de mensageria (RICHARDSON, 2018).

Entre as contramedidas descritas por Richardson (2018) estão bloqueios semânticos, atualizações comutativas, releitura com verificação e reorganização das etapas para reduzir riscos. Um bloqueio semântico representa uma condição de negócio, como uma ordem pendente que ainda não pode ser tratada como concluída. No caso de reservas, duas ordens concorrentes devem ser impedidas de comprometer o mesmo saldo ou os mesmos ativos além do limite permitido. Essa verificação deve ocorrer de maneira atômica no serviço responsável. A idempotência impede a repetição de um mesmo efeito, enquanto o controle de concorrência trata disputas entre operações distintas; ambos são necessários para avaliar a consistência.

1.6 Métricas de recuperação, desempenho e observabilidade

1.6.1 RECUPERAÇÃO E VERIFICAÇÃO DA CONSISTÊNCIA FINAL

A recuperação precisa ser observada pelo estado do negócio, além da ausência de erros técnicos. A distinção entre conclusão do fluxo e execução de compensações vem da formulação de Garcia-Molina e Salem (1987); em microsserviços, Richardson (2018) também destaca o estado dos participantes e a falta de isolamento. Neste estudo, a consistência final será entendida como o atendimento às regras definidas para ordens, reservas e liquidação após a conclusão ou recuperação do processo. Uma mensagem processada com sucesso não comprova, isoladamente, essa condição.

Para tornar o eixo principal verificável, a taxa de sucesso de compensação deverá relacionar as Sagas cuja recuperação terminou corretamente àquelas que exigiram compensação. O tempo de compensação deverá ter marcos explícitos, como o registro da decisão de compensar e a confirmação da última ação necessária. Esses marcos distinguem o tempo de detecção da falha do tempo gasto na compensação. Também será necessário registrar reservas remanescentes, efeitos duplicados e processos ainda pendentes ao final da observação. Trata-se da definição conceitual das medidas; prazos, cargas e critérios operacionais serão detalhados no protocolo experimental.

1.6.2 LATÊNCIA, PERCENTIS E VAZÃO

A latência de uma operação representa o tempo entre marcos definidos de início e término. Em um fluxo assíncrono, receber a confirmação de aceitação de uma ordem não significa que a Saga terminou. Por isso, o tempo de resposta inicial e o tempo até o estado final devem ser diferenciados. O *throughput*, ou vazão, representa a quantidade de operações concluídas por unidade de tempo; sua interpretação exige indicar se a contagem abrange conclusões de negócio, cancelamentos ou apenas requisições recebidas (RICHARDSON, 2018; KRISTIANTO; ZAHRA, 2021).

Dean e Barroso (2013) discutem como requisições excepcionalmente lentas podem afetar serviços distribuídos, fundamentando a atenção à latência de cauda. Os percentis descrevem posições na distribuição dos tempos observados: p50 corresponde à mediana; p95 e p99 indicam os limiares abaixo dos quais se encontram, respectivamente, aproximadamente 95% e 99% das observações. Eles não são percentuais de aumento nem probabilidades de sucesso. Por exemplo, se p99 de um conjunto de operações concluídas for 800 ms, aproximadamente 99% dessas observações terão duração até esse valor. O número é apenas ilustrativo, sem representar resultado deste trabalho.

Os percentis complementam a média porque permitem observar a parte mais lenta da distribuição. Entretanto, calculá-los apenas para operações concluídas pode ocultar

processos que permaneceram sem término durante uma falha. Assim, processos pendentes e limites de observação deverão ser relatados separadamente. O desempenho será interpretado em conjunto com a correção da recuperação, e não como substituto dela. A comparação preservará regras, cargas e condições de execução comparáveis, com conclusões restritas ao cenário investigado.

1.6.3 RASTREAMENTO DISTRIBUÍDO E COLETA DE MÉTRICAS

A observabilidade fornece informações para compreender o comportamento do sistema por meio de registros de eventos, métricas e rastros de execução. O rastreamento distribuído conecta operações realizadas em serviços diferentes; um *trace* representa a execução acompanhada e um *span* representa uma operação instrumentada dentro dela. Esses conceitos permitem relacionar as atividades dos participantes ao fluxo completo, em vez de examinar apenas registros isolados (RICHARDSON, 2018). No exemplo da ordem, essa correlação ajuda a distinguir o período de espera por uma reserva do período de execução de sua compensação.

O tratamento da latência de cauda em Dean e Barroso (2013) complementa as medidas de desempenho empregadas nos estudos comparativos já apresentados. Para a comparação proposta, a coleta deverá preservar informações suficientes para relacionar os tempos aos estados de negócio e às etapas de recuperação. Instrumentação comparável nas duas variantes e identificação dos marcos de medição serão necessárias para interpretar as observações do fluxo completo. As ferramentas e os procedimentos de coleta serão especificados no protocolo experimental.

Esta fundamentação reúne os conceitos que sustentam a futura comparação entre as duas formas de coordenação. Os exemplos apresentados são conceituais e esta entrega não contém resultados experimentais próprios.

1.7 Declaração sobre o uso de Inteligência Artificial

Declaro que utilizei o ChatGPT, da OpenAI, como apoio à organização da fundamentação teórica, à consulta e análise das referências e à revisão textual e dos arquivos LaTeX e BibTeX desta entrega. A definição do tema e o direcionamento do trabalho foram realizados por mim, e assumo a responsabilidade pelo conteúdo apresentado.

REFERÊNCIAS

AYDIN, S.; ÇEBI, C. B. Comparison of choreography vs orchestration based saga patterns in microservices. In: *International Conference on Electrical, Computer and Energy Technologies (ICECET)*. Praga: IEEE, 2022. p. 1–6. Disponível em: <https://doi.org/10.1109/ICECET55527.2022.9872665>. Citado na página 7.

DEAN, J.; BARROSO, L. A. The tail at scale. *Communications of the ACM*, v. 56, n. 2, p. 74–80, 2013. Disponível em: <https://doi.org/10.1145/2408776.2408794>. Citado 2 vezes nas páginas 10 e 11.

GARCIA-MOLINA, H.; SALEM, K. Sagas. In: *ACM SIGMOD International Conference on Management of Data*. ACM, 1987. p. 249–259. Disponível em: <https://doi.org/10.1145/38713.38742>. Citado 2 vezes nas páginas 5 e 10.

GRAY, J. Notes on data base operating systems. In: *Operating Systems: An Advanced Course*. Berlin, Heidelberg: Springer, 1978, (Lecture Notes in Computer Science, v. 60). p. 393–481. Disponível em: <https://doi.org/10.1007/3-540-08755-9_9>. Citado na página 4.

KRISTIANTO, H.; ZAHRA, A. Performance analysis of choreography and orchestration in microservices architecture. *Journal of Theoretical and Applied Information Technology*, v. 99, n. 18, p. 4220–4230, 2021. Disponível em: <https://www.jatit.org/volumes/Vol99No18/4Vol99No18.pdf>. Acesso em: 5 out. 2026. Citado 3 vezes nas páginas 4, 7 e 10.

MARQUES, D. M. S. *Microservices orchestration vs. choreography: a comparison and analysis*. Dissertação (Mestrado em Engenharia Informática) — Escola Superior de Tecnologia e Gestão, Instituto Politécnico do Porto, Porto, 2024. Disponível em: <https://hdl.handle.net/10400.22/27107>. Acesso em: 5 out. 2026. Citado 2 vezes nas páginas 4 e 8.

OVERLEAF. *Tutorials - Overleaf, Editor LaTeX*. 2025. Disponível em: <https://pt.overleaf.com/learn/latex/Tutorials>. Citado na página 2.

RICHARDSON, C. *Microservices Patterns*: With examples in Java. Shelter Island: Manning Publications, 2018. ISBN 9781617294549. Citado 9 vezes nas páginas 3, 4, 5, 6, 7, 8, 9, 10 e 11.

RUDRABHATLA, C. K. Comparison of event choreography and orchestration techniques in microservice architecture. *International Journal of Advanced Computer Science and Applications*, v. 9, n. 8, p. 18–22, 2018. Disponível em: <https://doi.org/10.14569/IJACSA.2018.090804>. Acesso em: 5 out. 2026. Citado na página 7.